menler

Your turning point in the AI era.

Claude Code

The Compact Playbook

SECTION 01

What Claude Code is

Claude Code is Anthropic's agentic coding tool. You describe what you want, and Claude reads your codebase, edits files across the project, runs commands, runs tests, and ships the change. Where regular Chat answers a question and Cowork hands off a file-based task, Claude Code hands off an engineering task.

It lives in your terminal by default, but the same engine also runs in VS Code, JetBrains IDEs, the Claude Desktop app, the web, Slack, and GitHub Actions. Your project memory and settings follow you across all of them.

Who Claude Code is built for

Claude Code is built for anyone who reads or writes code as part of their work. It scales from one-line bug fixes to multi-file refactors and full feature builds.

- **Software engineers** — feature work, refactors, debugging, code review
- **Senior engineers and tech leads** — running multiple agents in parallel across a large codebase
- **DevOps and platform teams** — automating CI, dependency updates, migrations
- **Technical PMs and ops engineers** — building internal tools, scripts, dashboards
- **Founders and indie hackers** — shipping prototypes and side projects faster
- **Non-engineers building tools** — describing what you want in plain language and getting working software back

You don't need to be a senior engineer to use it, but you do need to be willing to read code. Claude Code shows you what it changed — your job is to decide whether to keep it.

What it produces

- Working code changes across multiple files, ready for review
- Tests written, run, and fixed until the suite passes
- Git commits, branches, and pull requests with descriptive messages
- Refactors and migrations spanning thousands of lines
- Bug fixes that trace through the codebase to the root cause
- Documentation, READMEs, and release notes derived from the code itself

SECTION 02

Chat vs Cowork vs Claude Code

Anthropic has three main ways of working with Claude. Picking the right one for the job saves a lot of friction.

	CHAT	COWORK	CLAUDE CODE
Best for	Quick questions, single-step tasks, brainstorming	Multi-step knowledge work with files and outputs	Codebases, engineering tasks, real shipped code
Audience	Anyone	Non-developers and developers alike	Developers (and increasingly non-engineers building tools)
File access	No — you paste content in	Yes — local files, in a sandbox VM	Yes — full project access on your machine
Runs how long?	One turn at a time	Long-running, multi-step	Long-running, multi-step, can run multiple agents in parallel
Where it lives	Web, mobile, desktop	Desktop app only	Terminal, IDE, desktop, web, Slack, CI
Mental model	Asking a smart person a question	Handing a project to an assistant	Pairing with a senior engineer

A simple rule

Need an answer or a draft? → Chat
Need work done on your files? → Cowork
Need code shipped? → Claude Code

SECTION 03

Getting access

Requirements

- **A terminal** (macOS, Linux, WSL, or Windows) — or VS Code / JetBrains / Claude Desktop / a browser
- **A paid Claude plan** (Pro, Max, Team, or Enterprise) OR an Anthropic Console account with credits
- **A code project to work in** Claude Code reads what's in the current directory
- **Git is recommended** Claude Code uses it heavily for commits, branches, PRs

Install Claude Code (terminal)

Pick the method that fits your machine. The native installer is recommended — it auto-updates in the background.

MACOS, LINUX, WSL — NATIVE INSTALL (RECOMMENDED):

```
curl -fsSL https://claude.ai/install.sh | bash
```

WINDOWS POWERSHELL:

```
irm https://claude.ai/install.ps1 | iex
```

MACOS — HOMEBREW:

```
brew install --cask claude-code
```

WINDOWS — WINGET:

```
winget install Anthropic.ClaudeCode
```

Then go to your project and run claude:

```
cd your-project
claude
```

You'll be prompted to log in on first use. Once logged in, your credentials are stored and you don't have to log in again.

> **WARNING**
> Homebrew and WinGet installs do not auto-update. Run brew upgrade claude-code or winget upgrade Anthropic.ClaudeCode periodically to stay on the latest version.

Other surfaces (same engine, different home)

- **VS Code / Cursor:** search for "Claude Code" in Extensions. Inline diffs, plan review, and conversation history live in the editor.
- **JetBrains IDEs:** install the Claude Code plugin from the JetBrains Marketplace (IntelliJ, PyCharm, WebStorm, etc.).
- **Claude Desktop app:** install from claude.com/download, then click the Code tab. Useful for visual diff review and running multiple sessions side by side.

- **Web (claude.ai/code):** no install needed. Kick off long-running tasks in the cloud, run several in parallel, and check back when done.
- **Slack:** mention @Claude in a channel with a bug report or task, and get a pull request back.
- **CI/CD:** GitHub Actions and GitLab CI/CD integrations automate PR review, issue triage, and overnight fixes.

Your CLAUDE.md files, settings, and MCP servers work the same across every surface. Pick the one that fits the moment.

SECTION 04

The 5-step flow

Every Claude Code task follows the same pattern. Once you've done it once, the rest is variation.

Step 1

Open Claude Code in your project

Open your terminal, cd into the project, and run:

```
claude
```

Claude Code reads your project files as needed — you don't have to paste code in or manually add context. Type /help any time to see commands.

Step 2

Describe what you want

Tell Claude the outcome you need. Don't worry about specifying every file or step — it can find what it needs.

Weak: *"fix the bug"*

Strong: *"There's a bug where users see a blank screen after entering wrong credentials. Find the root cause and fix it. Add a test that catches the regression."*

Step 3

Review Claude's plan

For non-trivial tasks, Claude Code shows you its plan before executing. Read it. If something is off, say so before it runs:

"Don't touch the legacy auth code — that's getting deprecated next sprint."

"Use the existing utils/validation.ts module instead of writing new helpers."

"Run the test suite before you commit anything."

Step 4

Approve changes as they happen

By default, Claude Code asks for permission before editing files or running commands. You can approve each change, or switch to a more permissive mode for a session (covered in Section 6).

TIP
Press Shift+Tab to cycle through permission modes during a session.

Step 5

Review the diff, commit, and ship

When Claude is done, it shows you what changed. Review the diff like you would a teammate's pull request. Then ask Claude to commit, push, and open a PR:

```
commit my changes with a descriptive message and open a PR
```

If something is wrong, tell Claude what to fix. The session stays open — you don’t have to start over.

SECTION 05

CLAUDE.md and memory

Out of the box, Claude Code doesn't know your coding standards, your architecture choices, or how your team works. CLAUDE.md fixes that — and it persists across sessions.

What CLAUDE.md is

A plain markdown file you add to your project root. Claude Code reads it at the start of every session. It's where you tell Claude the things you'd want any new engineer on your team to know on day one.

What to put in it

- **Coding standards** (style guide, naming conventions, testing requirements)
- **Architecture decisions** (what's where, why it's structured that way)
- **Preferred libraries and patterns** "use Zod for validation, not Joi"
- **Build and test commands** specific to this project
- **Review checklist** things to verify before opening a PR
- **What's off limits** legacy modules, generated files, vendored code

Example CLAUDE.md

```
# Project: Acme API
## Stack
- TypeScript, Node 20, Fastify
- Postgres via Prisma
- Tests with Vitest
## Conventions
- Use Zod for input validation (not Joi or Yup)
- Route handlers go in src/routes, business logic in src/services
- No `any` types — prefer `unknown` and narrow
## Commands
- npm run dev — start the dev server
- npm run test — run the full test suite
- npm run lint:fix — auto-fix lint errors
## Don't touch
- /legacy — deprecated, scheduled for removal
- /generated — auto-generated from the schema
```

Auto memory

Claude Code also builds memory as it works — things like build commands it discovered, debugging insights, and gotchas it ran into. This auto-memory persists across sessions without you writing anything.

> **TIP**
> *Every time Claude makes a wrong assumption, add the correction to CLAUDE.md. Within a few weeks it becomes the user manual for working on your codebase — and output quality goes up sharply for everyone using it, including new humans.*

SECTION 06

Permission modes

Claude Code defaults to cautious: it asks for permission before editing files or running commands. You can dial that up or down depending on how much you trust the task.

MODE	WHEN TO USE IT
Default (ask first)	Every file edit and command needs your approval. Best for unfamiliar codebases, sensitive code, or first time using Claude on a project.
Accept edits	Auto-approve file edits in the current session but still ask before running commands. Good middle ground for well-scoped tasks.
Accept all	Claude can edit files and run commands without asking. Use for tasks where you want to walk away — overnight migrations, bulk refactors. Pair with version control.
Plan mode	Claude proposes a full plan but makes no changes. Best for complex tasks where you want to see the approach before any code is written.

TIP
Press Shift+Tab in any Claude Code session to cycle through modes.

WARNING
Permissive modes are powerful but irreversible mistakes are still possible. Always work on a branch you can throw away, and never run Accept All against a production system or an uncommitted main branch.

SECTION 07

Skills, hooks, and MCP

Claude Code is built to extend. The three big extension points are skills, hooks, and MCP — each solves a different problem.

Skills

Skills are packaged workflows your team can share. Think of them as named recipes. A /review-pr skill might run lint, tests, and a security pass against the current branch. A /deploy-staging skill might cut a release, push it, and post to Slack.

Type / inside Claude Code to see all available skills and slash commands.

Hooks

Hooks run shell commands before or after Claude Code does something. Use them to auto-format after every edit, run lint before a commit, or block edits to certain files entirely. Hooks are how teams enforce standards without nagging.

MCP — Model Context Protocol

MCP is an open standard for connecting Claude Code to external tools and data sources. With MCP, Claude can read your Jira tickets, query your Postgres instance, look up design docs in Google Drive, or use any custom internal tool your team has built.

COMMONLY USED MCP SERVERS

- **GitHub / GitLab** issues, PRs, repositories
- **Jira / Linear** read and update tickets
- **Postgres / Snowflake** query databases directly
- **Slack** pull threads, post messages
- **Google Drive / Notion** read design docs and specs
- **Sentry / Datadog** pull errors and logs into investigations
- **Custom MCP servers** connect to your team’s internal tools

WARNING
MCP servers get access to your data in those services. Only enable servers from trusted sources, and review what they can read and write before granting access.

SECTION 08

Sub-agents and parallel work

For big tasks, Claude Code can spawn multiple agents that work on different parts of the problem simultaneously. A lead agent coordinates, hands subtasks to specialist agents, and merges results.

This is how teams pull off the eye-popping speed gains you read about — a Scala-to-Java migration in four days, a 50,000-line Python-to-Go port in twenty hours. The work parallelises.

Two ways to run parallel work

- **Sub-agents within a session** the lead agent assigns subtasks (“agent 1: migrate the auth module; agent 2: update the tests; agent 3: update docs”) and merges the output.
- **Background agents** run several full Claude Code sessions in parallel and watch them from one screen. Useful for trying multiple approaches to the same problem and picking the best one.

When to reach for parallelism

- Migrations and ports across many files where work decomposes cleanly
- Bulk refactors — apply the same change pattern across the codebase
- Trying two or three approaches to a hard design problem in parallel
- Long-running tasks where you don’t want to block on one

NOTE
Don’t reach for sub-agents on small tasks. The orchestration overhead outweighs the benefit. Single-agent Claude Code is faster for anything under an hour of work.

SECTION 09

Safety — what to know before you let it run

Claude Code can make real changes to your code, your filesystem, and the services your MCP servers connect to. The safety design is built around three things: permission prompts, your version control, and your active review.

Permission prompts are your friend

Default mode asks before every file edit and every command. For unfamiliar tasks, leave it on. The friction is the point — it forces you to see what's happening.

Version control is non-negotiable

Always work on a branch. Always commit before you let Claude make big changes. Git is your undo button — without it, you have none. Claude Code is good at git itself, so if you're not in the habit of branching, ask Claude to do it:

```
create a branch called feature/refactor-auth and switch to it
```

Review the diff before merging

Claude Code is fast. It can produce hundreds of lines of changes in minutes. Treat its output like a pull request from a new teammate — read it, run the tests, look for surprises, then merge. Don't skip the review just because the tests pass.

What Claude Code is NOT suitable for

- Running unattended against production systems
- Editing infrastructure-as-code without a plan review and human approval
- Handling secrets, API keys, or PII you don't want models to see
- Codebases you don't have backups or version control for
- Tasks where you don't understand the domain well enough to review the output

WARNING
Anthropic's safety stance: humans decide what ships. Claude Code's defaults err on the side of asking. Don't disable those defaults just to move faster — disable them only when you've decided the task is safe.

SECTION 10

Ten use cases with starter prompts

Copy, adapt, paste into Claude Code. Pick the one closest to your real work first.

1 Onboard yourself to an unfamiliar codebase

FOR *anyone joining a new project or inheriting a repo*

STARTER PROMPT

Read this codebase and explain it to me like I just joined the team. Cover:
1. What the project does and the main user-facing flows
2. The high-level architecture and how the major modules connect
3. The tech stack and the unusual or non-obvious choices
4. Where the entry points are (server, CLI, jobs, etc.)
5. The 5 files I should read first to understand how things hang together

2 Fix a bug from a description or error

FOR *anyone with a bug report and not enough time*

STARTER PROMPT

There's a bug where uploading a file over 10MB silently fails — no error, no message, the upload modal just closes. Trace it through the codebase, find the root cause, and fix it. Add a test that catches the regression. Open a PR when done.

3 Build a feature end-to-end

FOR *engineers shipping new functionality*

STARTER PROMPT

Add a 'saved searches' feature. Users should be able to:
1. Save the current filter state with a name
2. See a list of saved searches in the sidebar
3. Click one to re-apply those filters
4. Rename or delete saved searches

Match the existing code patterns. Add tests. Update the README.

4 Refactor without breaking anything

FOR *anyone cleaning up old code*

STARTER PROMPT

Refactor the auth module from callbacks to async/await. Don't change behavior, don't change the public API. Run the test suite before and after — make sure the same tests pass. Commit in small chunks so I can review.

5 Migrate or port at scale

FOR *platform teams and tech leads*

STARTER PROMPT

Port the /scripts directory from Python to TypeScript. Match the existing TS code style in /src. Use sub-agents to parallelise across files. After each script is ported, run it against the same test fixtures and verify the output matches. Stop and ask me if any script does something you can't preserve cleanly.

6 Write tests for untested code

FOR *anyone with low coverage and a backlog*

STARTER PROMPT

Look at /src/services/billing.ts. It has no tests. Write a thorough test suite: happy path, edge cases, error paths. Match the testing patterns used in the rest of the project. Run them and fix any bugs you discover along the way.

7 Review your own pull request

FOR *engineers before they ask for review*

STARTER PROMPT

Look at my current branch's diff against main. Review it as a senior engineer:
– Logic and edge case issues
– Performance and security concerns
– Code style and consistency with the rest of the codebase
– Missing tests
– Anything that looks like a copy-paste mistake
Be honest. I'd rather hear it now than from my reviewer.

8 Investigate a CI failure

FOR *anyone staring at a red build*

STARTER PROMPT

The build is failing on the main branch. Pull the latest CI logs, find the failing tests, figure out why they're failing, and fix them. If it's a flake, mark it as such and explain. If it's a real regression, trace it to the commit that introduced it. Open a PR with the fix.

9 Update dependencies safely

FOR *anyone dreading the next dependency bump*

STARTER PROMPT

Update all dependencies to their latest non-breaking versions. For each one:
1. Check the changelog for breaking changes
2. Skip anything with a major version bump — flag those for me separately
3. Update, run the tests, fix anything broken
4. Commit one dependency at a time so I can revert if needed

10 Build a working prototype from a description

FOR *founders, indie hackers, and non-engineers*

STARTER PROMPT

Build a simple web app for tracking my reading list. Features:
– Add a book (title, author, status: to-read/reading/done)
– Mark a book as done with a rating and a one-line note
– See all books filtered by status
Use whatever stack is fastest to ship and easy to host. Add a README explaining how to run it. I'll worry about deployment once I see it working locally.

SECTION 11

Pro tips

Describe the outcome, not the steps

Claude Code plans the implementation itself. Telling it “first edit this file, then add this import” usually produces worse output than describing what “done” looks like and letting Claude figure out how to get there.

Let it explore before changing anything

Before any non-trivial task, ask Claude to read the relevant code first. “Read the authentication module and tell me how it works” before “refactor the authentication module” produces dramatically better refactors.

Use plan mode for anything risky

Plan mode shows you the full approach with zero file changes. It’s the highest-leverage feature for big tasks — five minutes reading the plan saves an hour fixing wrong output.

Branch first, ask questions later

Ask Claude to create a branch for any task bigger than a one-line change. If something goes wrong, you can throw the branch away and start over.

Commit in small chunks

Tell Claude to commit each logical step rather than dumping everything in one giant commit. Smaller commits are easier to review, easier to revert, and easier to bisect when something breaks.

Build CLAUDE.md over time

Every time Claude makes a wrong assumption, add the correction to CLAUDE.md. Within a few weeks your CLAUDE.md becomes the user manual for working on your codebase — and quality goes up sharply for everyone using it, including new humans.

Use slash commands and skills

Press / to see all available commands and skills. /clear resets context when a conversation gets too long. /resume picks up where you left off. /model lets you switch between Claude models mid-task.

Pipe it into your existing workflows

Claude Code is composable — a Unix-friendly CLI, not a special UI you have to live inside. Pipe log output for anomaly detection, diff output for security review, or any shell stream into Claude with -p.

Schedule the recurring stuff

Use /schedule or routines for things you do every day or every week: morning PR reviews, overnight CI failure analysis, weekly dependency audits. Routines run on Anthropic’s infrastructure so they keep running even when your machine is off.

Parallelise only when the work decomposes

Sub-agents are amazing for migrations, ports, and bulk refactors. They’re a tax on small tasks. Don’t reach for them by default.

SECTION 12

Troubleshooting

PROBLEM	WHAT TO DO
Claude can’t find a file or command	Make sure you ran claude from the project root, not from /. Claude reads from the current directory tree. Use ls to confirm the project structure is what you expect.
Output keeps missing the mark	The issue is almost always the prompt or the missing context. Add a CLAUDE.md, point at relevant files explicitly, and describe the outcome more precisely. “Add error handling” is vague — “Add error handling that returns a 422 with the Zod error message when validation fails” is actionable.
Permission prompts every few seconds	Switch to Accept edits mode with Shift+Tab. For tasks where you trust the scope completely, Accept all — but only on a branch you can throw away.
Hitting context window limits	Run /clear to reset conversation history mid-task once the current step is done. Or break the task into smaller pieces. Long sessions accumulate tokens fast.
Wrong direction completely	Stop the task with Ctrl+C. Start over with a sharper prompt. Don’t try to redirect mid-task on something big — the plan was wrong from the start.
Tests pass but the change is wrong	Trust your reading of the diff more than the test suite. Tests verify what they test — they don’t verify what they don’t test. Ask Claude to write an additional test that would have caught the issue.
Login or auth errors	Run /login to re-authenticate. If you’re using an API key via ANTHROPIC_API_KEY, note that some features only work with subscription login, not API keys.
Claude Code on Windows feels off	Install Git for Windows so Claude Code can use the Bash tool. Without it, it falls back to PowerShell and some commands behave differently. WSL is a smoother experience overall.
Sub-agents are slower than one agent	You probably reached for them on a task that doesn’t decompose. For anything under an hour of work, single-agent is faster. Reserve sub-agents for migrations and bulk changes.

SECTION 13

Your first 30 minutes

If you've never used Claude Code, here's the fastest way to get from zero to confident.

Minute 0–5 — Set up

- Install via the native installer (or Homebrew/WinGet)
- Pick a low-stakes project — a side project or scratch repo, not production
- cd into the project and run claude — log in when prompted

Minute 5–10 — Get a feel for it

- Type: what does this project do?
- Read Claude's answer. This is how it sees your codebase. Try a few more:
- what technologies does this project use?
- where is the main entry point?
- explain the folder structure

Minute 10–15 — Write a CLAUDE.md

- Create a CLAUDE.md in the project root. Keep it short — 5–10 lines is enough to start.

Minute 15–25 — Make a real change

- Make a branch first, then ask for something small and concrete:
- create a branch called experiment/claude-code-tour and switch to it
- add a /health endpoint that returns { status: 'ok', version: <package.json version> }. follow the existing route conventions. add a test for it.
- Read the plan. Approve each change. Read the diff. Run the test.

Minute 25–30 — Ship and reflect

- Ask Claude to: commit the changes with a descriptive message and push the branch
- Now think about what you'd hand off next. Make a list of 3 tasks from your real work that would be good Claude Code candidates:
- Well-scoped (a clear definition of done)
- Multi-step (would take you an hour or more by hand)
- Reviewable (you can tell good output from bad)

– Pick the highest-pain one. That’s your next Claude Code session.

```
# Project: [name]
## Stack
- [language, framework, database]
## Commands
- [how to run dev]
- [how to run tests]
## Conventions
- [one or two style rules]
## Don't touch
- [legacy or generated folders]
```

TIP
That’s it. You now have Claude Code installed, a CLAUDE.md written, and a real change shipped. Everything else is practice.

Quick Reference Card

THE MENTAL MODEL

Chat = ask a question, get an answer
Cowork = hand off a file-based task, get finished work
Claude Code = pair with a senior engineer

THE 5-STEP FLOW

– Open Claude Code in your project (claude)
– Describe the outcome you want
– Review the plan before it executes
– Approve changes as they happen
– Review the diff, commit, ship

PROMPT FORMULA

[What you want done].
[Constraints — don’t touch X, match Y patterns].
[Definition of done — tests pass, PR opened, etc.].

ESSENTIAL COMMANDS

claude start an interactive session
claude "task" run a one-off task
claude -p "query" quick query then exit (pipeable)
claude -c continue most recent conversation
/clear reset conversation history
/login switch accounts
/help see all commands
Shift+Tab cycle permission modes

HIGHEST-LEVERAGE SETUP STEPS

– Write a CLAUDE.md (10 minutes, every future session benefits)
– Branch before any non-trivial task (git is your undo button)
– Use plan mode for anything you’re not sure about
– Schedule the recurring stuff (morning PR reviews, dependency audits)

Your turning point in the AI era.

Claude Code · The Compact Playbook · menler.in